

AI for Medicine – Project Report

Prof. Stefano Diciotti – University of Bologna

Academic Year: 2025-2026

Student Name: Dilshod Mirakhmadov

Degree Program: Electronic Engineering

Submission Date: 20/06/2026

Note: The report should be concise but complete. Typical length: 8–10 pages.

1. Project Title

Classification of Breast Cancer Diagnosis (Malignant vs. Benign) Using Supervised Machine Learning with nested Cross-Validation scheme

2. Problem Statement

Describe the medical or biological problem addressed by this project. Why is it relevant? What is the scientific or clinical motivation?

My project addresses the medical diagnosis of breast cancer in women, which is one of the cancer types with a high rate of lethal outcomes and can also have a big impact on other parts of the human body, even after proper medical treatment. This is why early and accurate diagnosis is so important: a person who is diagnosed early may still be able to live a full life.

3. Objective of the Study

State the specific objective. What are you trying to predict, classify, segment, or analyze?

The objective is to binary classify two classes – Benign(safe) vs Malignant(dangerous)

We try to predict using 30 numerical features extracted from images of cell nuclei such as radius, texture, perimeter, area, smoothness and others – for mean, error and worst values.

Specific goal of my project is to build and compare 4 classification models such as Logistic Regression, Random Forest, SVM-support Vector Machine and KNN after choose the best model by fair evaluation procedure based on Nested Cross-Validation.

I tried to obtain a model with strong generalization ability without overfitting and also ensuring that pipelines are fully protected against data leakage and the code is fully reproducible

4. Dataset Description

Name and source of the dataset (e.g., MedMNIST, PhysioNet, Kaggle).

- *Number of samples*
- *Type (image, signals, tabular data) and description of data*
- *Available labels (diagnosis, classes, target variables)*
- *Issues such as class imbalance or noise*

In the project used **Wisconsin Diagnostic Breast Cancer (WDBC)** dataset

The one that commonly used for breast cancer classification tasks. The dataset was obtained from Kaggle (Breast Cancer Wisconsin Diagnostic Data Set).

- The dataset contains 569 samples, each corresponding to a single patient. The data is tabular, with all features represented as numerical values describing characteristics of cell nuclei observed in breast tissue samples.
- Each sample has 30 numerical features, based on 10 basic characteristics (radius, texture, perimeter, area, smoothness, compactness, concavity, concave points, symmetry, and fractal dimension). For each characteristic, three values are provided: the mean, the standard error, and the worst value, giving a total of 30 features.
- The target variable is the diagnosis, which is binary: Malignant (M) or Benign (B). Out of 569 samples, 357 are Benign and 212 are Malignant, showing a moderate class imbalance that was considered during data splitting and evaluation (see Section 5).
- A first check of the dataset showed no missing values and no duplicate samples, so no extra cleaning was needed before preprocessing.

5. Data Preprocessing

Detail how the data was processed before training the model:

- *Data cleaning, normalization, dimensionality reduction, etc.*

Before training the models, the dataset was split into a training set (80%) and a test set (20%).

I used a stratified split, which means the proportion of Malignant and Benign cases stays the same in both the training and test sets. The test set was kept aside and was not used again until the final evaluation of the models in Section 8.

Since the 30 features have very different scales (for example, area values are around 1000, while smoothness values are around 0.1), we applied a StandardScaler to normalize all the features. This transformation makes every feature have a mean of 0 and a standard deviation of 1, which helps models like Logistic Regression, SVM, and KNN to work better.

It is important to mention that the scaler was fitted only on the training data, and then applied to the test data using the same parameters. This was done to avoid any data leakage, as explained in more detail in Section 6. Inside the Nested Cross-Validation procedure, this normalization step was implemented using sklearn Pipeline, which automatically applies the scaler correctly on each fold.

No dimensionality reduction technique (such as PCA) and no feature selection were used in this project; all 30 original features were kept for training the models.

6. Avoiding Data Leakage

Explain how the pipeline avoids data leakage. For example:

- Proper separation of training and test data
- Feature selection and normalization are performed within the training folds
- No peeking at the test set during model design or tuning

Data leakage is one of the most important problems to avoid in a machine learning project, especially in medicine, because it can make a model look much better than it actually is. In this project, several steps were taken to prevent it:

First, it is important to mention that the test set was never used in tuning hyperparameters or in feature selection. It was only used in the final prediction stage (Section 8), after all modeling decisions had already been made.

- Second, the StandardScaler inside the sklearn Pipeline guarantees that normalization is performed separately on each fold of the Nested Cross-Validation, without biasing the validation fold with information from outside the training fold. In addition, the stratified split (80/20) tackles the problem of class imbalance in the data, ensuring that both the training and test sets are representative and reducing the risk of evaluation issues caused by an unbalanced split.

Also, the L1.4 checklist confirmed that there are no duplicate samples between the training and test sets. This means that every row in the dataset corresponds to a unique case, so there is no risk that the same case could appear in both sets.

Finally, the use of Nested Cross-Validation itself is a key part of avoiding data leakage during model selection. The inner loop was used only to tune hyperparameters, while the outer loop was used only to evaluate the model. Because these two steps were kept completely separate, the final performance estimate is unbiased and not artificially inflated by the hyperparameter tuning process.

7. Machine Learning Pipeline

- Models used (e.g., Random Forest, CNN, Logistic Regression)
- Validation techniques (e.g., k-fold, stratified hold-out)
- Evaluation metrics (accuracy, AUC, precision/recall, etc.)
- Regularization, feature selection, hyperparameter tuning

Four different classification models were trained and compared: Logistic Regression, Random Forest, Support Vector Machine (SVM), and K-Nearest Neighbors (KNN). These four models were chosen because they work in very different ways - Logistic Regression is a simple linear model, Random Forest is an ensemble of decision trees, SVM tries to find the best boundary between classes, and KNN makes predictions based on the closest data points. Comparing models with different approaches gives a better idea of which type of model fits this dataset best.

Each model was built using sklearn Pipeline, which combines two steps into one object: a StandardScale and the classifier itself. Using a Pipeline is important because it makes sure the normalization step is always done correctly — the scaler is fitted only on training data and never sees the validation or test data in advance.

To evaluate the models fairly, Nested Stratified Cross-Validation was used. This method has two loops:

The outer loop (5-fold Stratified K-Fold) splits the data into 5 parts and is used to honestly evaluate how well each model performs.

The inner loop (3-fold Stratified K-Fold, inside GridSearchCV) is used only to search for the best hyperparameters for each model.

Keeping these two loops separate is the key idea behind Nested CV: it prevents the model from being evaluated using the same data that was used to tune it, which would make the results look better than they really are.

For each model, GridSearchCV was used to test different hyperparameter values and find the best combination:

Logistic Regression: different values of C (the regularization strength)

Random Forest: different values of $n_estimators$ (number of trees) and max_depth (depth of each tree)

SVM: different values of C and different kernel types (linear and rbf)

KNN: different values of $n_neighbors$ (number of neighbors)

The parameter C , used in both Logistic Regression and SVM, controls the strength of regularization: a smaller C means stronger regularization (simpler model, less risk of overfitting), while a larger C means weaker regularization (more flexible model).

The main metric used to compare models was ROC AUC (Area Under the ROC Curve). This metric was chosen instead of simple accuracy because it is not affected by class imbalance — since our dataset has more Benign cases than Malignant ones, accuracy alone could give a misleading picture of how good the model really is.

After completing the Nested Cross-Validation and identifying the best-performing model, a final round of GridSearchCV was run on the training set to find the best hyperparameters one more time, and the final model was retrained on the entire dataset.

8. Results

Present key results (preferably in a table or with figures):

- ROC curve, confusion matrix, performance scores
- Comparison of different models, if applicable

The result of Nested CV (fair evaluation of each model before choosing final one)

Model	ROC AUC (Mean $\pm$ Std)
Logistic Regression	0.994 $\pm$ 0.010
Random Forest	0.988 $\pm$ 0.009
SVM	0.995 $\pm$ 0.006

KNN	0.988 ± 0.012
-----	-------------------

Best Hyperparameters (using GridSearch for every model)

Model	Best Hyperparameters
Logistic Regression	$C = 1$
Random Forest	$n_estimators = 100, max_depth = None$
SVM	$C = 1, kernel = 'rbf'$
KNN	$n_neighbors = 9$

Final results for test set

Model	AUC	Accuracy	Precision	Recall	F1-score
Logistic Regression	0.995	0.982	0.986	0.986	0.986
Random Forest	0.994	0.956	0.959	0.972	0.966
SVM	0.995	0.982	0.986	0.986	0.986
KNN	0.994	0.974	0.960	1.000	0.980

Figure 1. Confusion matrices for all four models on the test set.

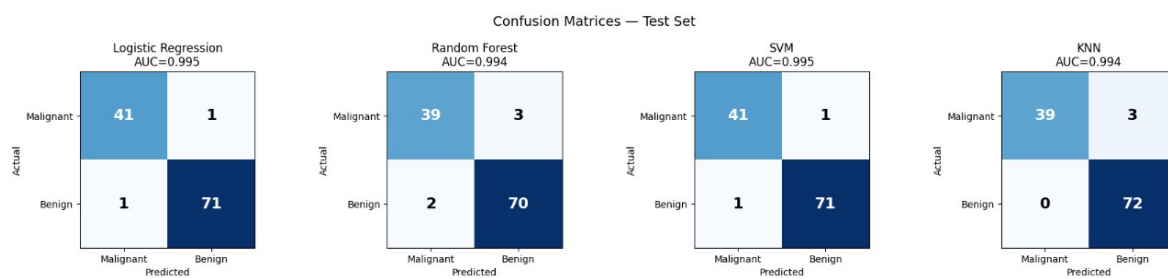


Figure 2. ROC curves for all four models on the test set.

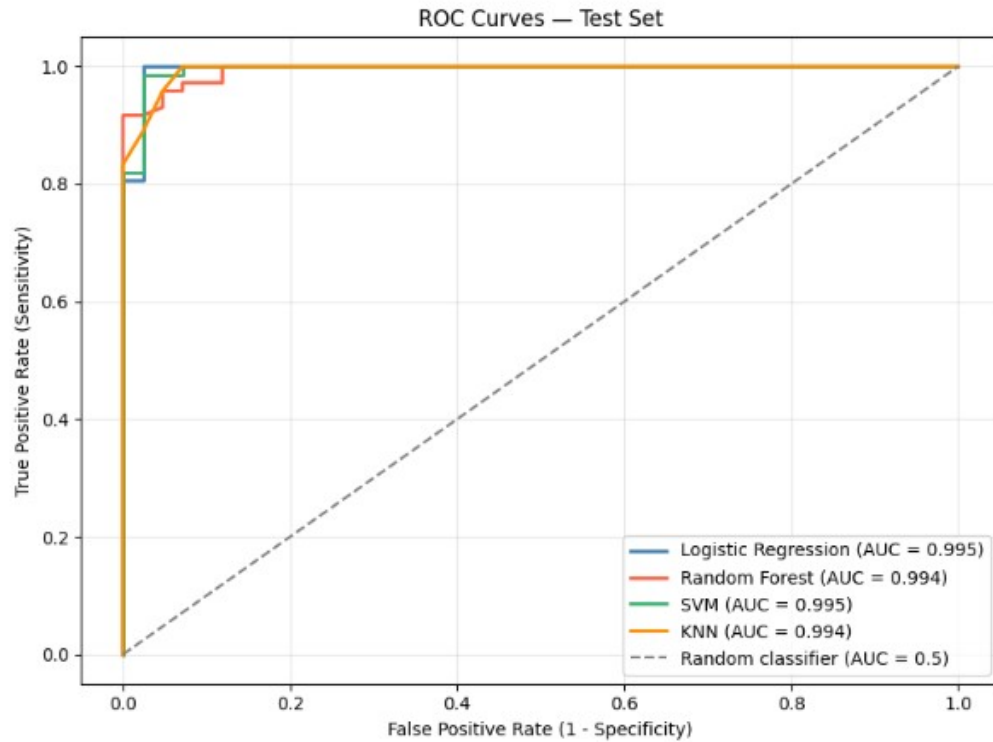
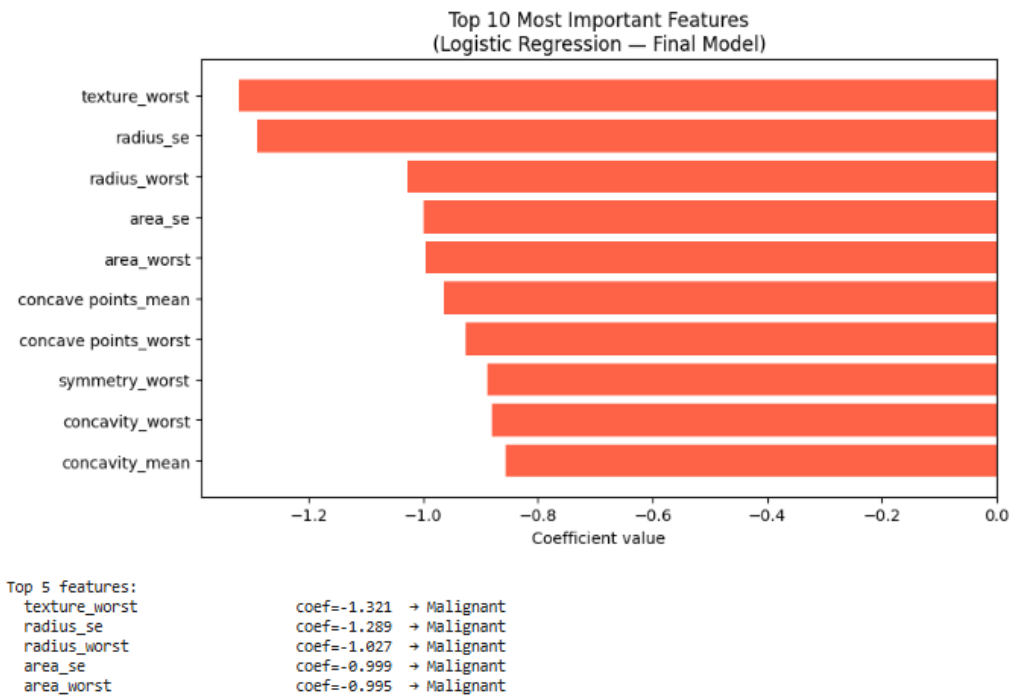


Figure 3. Top 10 most important features of the final Logistic Regression model.



9. Discussion

- *Interpretation of results*
- *Limitations*

Logistic Regression and SVM performed equally well having best result in $AUC = 0.995$

The fact that LR (simple linear model) performed on same level as SVM with non-linear kernel says in dataset classes separated well in linearly.

That is why the final model was chosen relying on this factor, which is Logistic Regression, because the interpretation of the result is easier (can be seen in Section 8 in Feature importance). Also, this model trains faster and has less risk of error when the model is retrained with new data.

If we look at the confusion matrices in Figure 1, it is important to think about the **false negatives**, meaning cases where a tumor is actually Malignant but the model predicted Benign. In a real clinical setting, this type of error is more dangerous than a false positive, because it could delay treatment for a patient who actually needs it. It is worth noting that KNN had a Recall of 1.000, meaning it did not miss any Malignant case in the test set. This could be seen as a clinically desirable property, even though its overall AUC was very slightly lower than Logistic Regression and SVM.

The ROC curves in Figure 2 show that all four models stay close to the top-left corner of the plot, which means all of them are able to separate the two classes very well. The differences between the models are small, confirming that this dataset is relatively easy to classify with standard machine learning methods.

The feature importance results in Figure 3 show that the most important features are mostly "worst" values (worst texture, worst radius, worst area), rather than mean values. This makes sense from a medical point of view, since the malignancy of a tumor is often driven by its most abnormal cells, not by the average behavior of all cells in the sample.

- When it comes to limitations of this project:
 - the dataset is relatively small, with only 569 samples, so the model may not generalize well to more diverse patient populations.
 - the data also comes from a single source, and no external validation on a different dataset was performed.
 - the features used in this project were already extracted from the original biopsy images, so the model works directly with numerical measurements rather than with raw images.

10. Conclusions and Future Work

- *Short conclusions*
- *Ideas for future improvement or next steps*

Main Conclusion is in the project we have built functional classifier that distinguishes two diagnoses of the Breast Cancer (Malignant(dangerous) vs Benign(safe)) with very high (using Logistic Regression model) prediction ($AUC=0.995$) that ensures simplicity in future exploitation and interpretation of the model

Note to mention the outcome of the Nested CV was fair without any data leakage and confirming the full reproducibility of the code.

For the development of this model:

- We can take another dataset from another source to be sure that scheme would work not only on «569 patients» ensuring no bias. Also, we can add more data, meaning that add features of the people with different age group and ethnic group)
- Try to employ image processing model that would significantly ease and accelerate process of diagnosis of the Breast Cancer

11. Ethics and Data Privacy

Describe how ethical and privacy considerations were addressed in the project.

- Confirm that the dataset used is publicly available and permitted for academic or research use.
- State any licenses or usage restrictions attached to the data or to any third-party code used.

If applicable, include a reference to the data source's license or terms of use.

The dataset used in this project, the Wisconsin Diagnostic Breast Cancer (WDBC) dataset, is publicly available through the Kaggle web-source, and it is commonly used for academic and research purposes in the machine learning community.

The data is fully anonymized: it does not contain any patient names, identification numbers, or other personal information.

The third-party code used in this project, mainly the scikit-learn library, is open-source and released under the BSD license, with no restrictions on academic use.

It is important to note that this project is intended for educational purposes. Even though the model achieved high performance, it should not be used as a real diagnostic tool without proper clinical validation, regulatory approval, and supervision by qualified medical professionals. Machine learning models like this one should be seen as a decision support tool, not as a replacement for a doctor's judgment.

12. Code and Reproducibility

- Name of the Python notebook (.ipynb) developed in Google Colab
- Any notes for reproducing the results (dependencies, instructions)

Breast_cancer_classification_nestedCV.ipynb which is developed in Google Colab environment

Dependencies on:

numpy 2.0.2
pandas 2.2.2
matplotlib 3.10.0
sklearn 1.6.1

The dataset, as mentioned before, was taken from the Kaggle web-source and was downloaded to my personal Google Drive (file_path = '/content/drive/MyDrive/Colab Notebooks/data.csv').

RANDOM_STATE = 42 was used in all train_test_split, StratifiedKFold, and models, ensuring my results are repeatable.

13. References

Include all relevant references cited in the report.

You can use any standard citation format (e.g., APA, IEEE) consistently throughout the document.

- Wolberg, W. H., Street, W. N., & Mangasarian, O. L. (1995). *Breast Cancer Wisconsin (Diagnostic) Data Set*. UCI Machine Learning Repository.
<https://archive.ics.uci.edu/dataset/17/breast+cancer+wisconsin+diagnostic>
- Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., ... & Duchesnay, E. (2011). Scikit-learn: Machine Learning in Python. *Journal of Machine Learning Research*, 12, 2825–2830.
- Diciotti, S. (2025–2026). *Artificial Intelligence for Medicine* [Course materials]. University of Bologna.
- Kapoor, S., & Narayanan, A. (2023). Leakage and the reproducibility crisis in machine-learning-based science. *Patterns*, 4(9), 100804.
- Carter, R. E., Attia, Z. I., Lopez-Jimenez, F., & Friedman, P. A. (2019). Pragmatic considerations for fostering reproducible research in artificial intelligence. *npj Digital Medicine*, 2(1), 1–4.